Matteo Bertuzzi

PERSONAL PROJECT | CASE STUDY

Weather-Adaptive Demand Forecasting

Stateful Agent System for Grocery Retail Replenishment

Author	Matteo Bertuzzi - AI Engineer
Illustrative Client	FreshMart Inc. (fictitious US grocery retailer)
Project Type	Personal project and portfolio case study
Technology	LangGraph, GPT-4o-mini, Open-Meteo, Google Sheets, FastAPI
Dataset	5,000-row synthetic grocery sales dataset (open-source)
Document Date	June 2026

I built this system to answer a specific question: if I give an agent a 7-day weather forecast and a year of sales history, can it produce a replenishment quantity a buyer would actually act on? Grocery retail is an interesting domain because the gap between what the data contains and what most forecasting tools use is unusually wide. Temperature and precipitation visibly shift demand across beverage, frozen, and produce categories, yet the industry default is still a rolling average that treats a forecast heatwave the same as last March. This case study documents what I built, how it works, what the synthetic dataset shows, and where the approach has real limits.

Executive Summary

This project implements a four-node LangGraph agent that produces weather-adjusted 7-day demand forecasts for grocery SKUs. The agent fetches a real-time weather forecast from the Open-Meteo API, queries historical sales across five analytical dimensions using GPT-4o-mini in tool-use mode, and passes the collected evidence to a synthesis agent that returns a predicted quantity and a confidence score. Forecasts are written back to a Google Sheets workbook via the gspread library. The full source code is available on GitHub.

Analysis of the synthetic dataset (5,000 rows, 58 SKUs, June 2024 through May 2025) shows weather-related demand variation across all tested categories. The temperature signal in the Drinks category is real but weak: a linear fit yields $R^2 = 0.03$, and the relationship is non-monotonic above roughly 18°C, where demand in the dataset declines rather than continuing to rise. The agent handles this better than a fixed coefficient would because it queries actual temperature-band records rather than applying a rule.

Financial and operational projections in this document are illustrative scenarios for a fictitious grocery retailer (FreshMart Inc., 187 stores). They are included to show how the system's outputs connect to replenishment economics, not to claim measured results. The system has not been deployed in a live store.

REAL in this document	ILLUSTRATIVE in this document
The LangGraph StateGraph architecture and conditional routing logic. The Open-Meteo API integration and weather parsing. The gspread data access layer and Google Sheets schema. The FastAPI endpoint. The 5,000-row synthetic dataset (grocery_sales.csv). The weather-demand bar chart (Figure 2) and temperature analysis (Figure 3), both computed on that dataset. $R^2 = 0.03$ for temperature vs Drinks demand.	FreshMart Inc. and all company figures (store count, revenue, headcount). The 18-store pilot scenario and any "control group" framing. All dollar projections in Figure 6. MAPE figures for Bakery, Fruit & Veg, and Snacks (labeled "projected" in Figure 5). The phrases "statistically significant" and "controlled pilot" do not appear in this document.

5,000	4	5	R^2=0.03
Rows in synthetic dataset (58 SKUs)	LangGraph agent nodes	Evidence query dimensions	Temperature signal (Drinks, linear fit)

Business Context

About FreshMart Inc. (illustrative scenario)

FreshMart Inc. is a fictitious mid-size US grocery retailer used as the operational backdrop for this case study. The scenario: 187 store locations across 12 states in the Midwest and Southeast, approximately $1.42 billion in annual revenue, and roughly 12,000 active SKUs per store across ten product categories. All specific figures are illustrative assumptions chosen to be plausible for a retailer of this type, not derived from any real company.

Why Weather Matters in Grocery

Grocery retail operates on net margins of 1 to 3%, making demand forecasting accuracy directly consequential. Perishable inventory converts from margin contributor to write-off quickly when overordered. Stockouts in a competitive market produce immediate substitution to a nearby store rather than a deferred purchase. Both failure modes have asymmetric costs at grocery scale.

Weather is a recognized demand driver in categories including beverages, frozen goods, produce, bakery, and personal care. Most mid-tier retailers address this with static seasonal indices, which capture calendar-level patterns but carry no intra-seasonal signal. A warm week in October looks the same as any other October in a rolling average.

Monthly Sales Volume (synthetic dataset, Jul 2024 - May 2025)

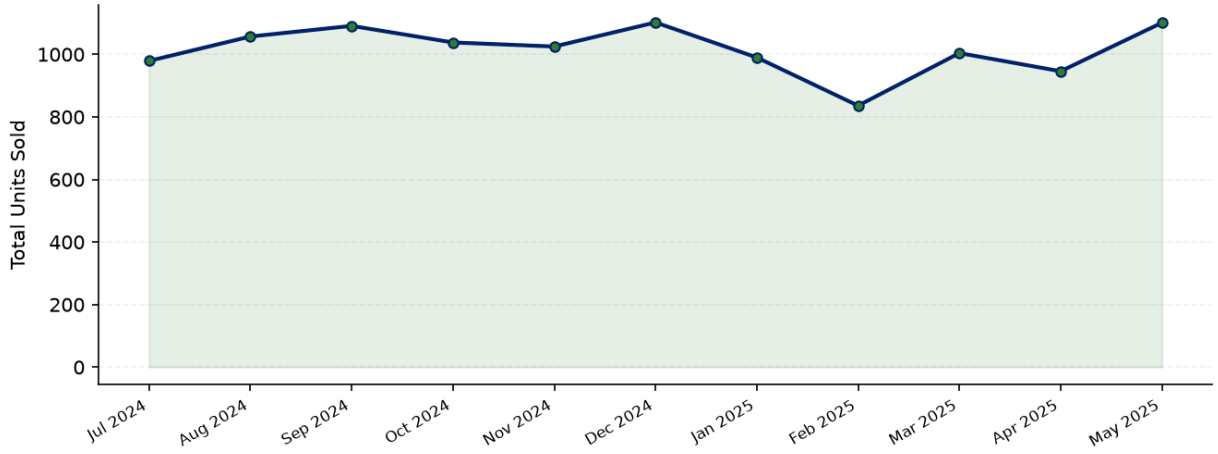

Figure 1. Monthly unit sales in the synthetic dataset across 58 SKUs, for the 11 complete calendar months from July 2024 through May 2025. The partial endpoint months (June 2024, 25 days; June 2025, 5 days) are omitted to avoid misleading month-end cliff artifacts.

Problem Statement

The Forecasting Gap This System Addresses

A typical mid-tier grocer's demand planning function uses a rolling 8-week moving average with a manually maintained seasonal index. Buyers adjust forecasts weekly, consuming substantial planning capacity. The process has four structural weaknesses that a weather-adaptive agent addresses directly:

- **No weather signal.** The model has no mechanism to adjust order quantities based on upcoming conditions, despite the relationships visible in the data.
- **Manual throughput ceiling.** Every forecast revision requires buyer intervention, creating a bottleneck that limits response speed.
- **Diluted recency.** An 8-week window under-weights recent demand signals. A cold spell immediately before a warm forecast period is absorbed into the average rather than flagged.
- **No confidence differentiation.** All SKU forecasts carry the same implicit confidence, preventing downstream systems from routing uncertain decisions to human review.

Illustrative Performance Gap (FreshMart scenario)

The following figures represent company-wide annual baselines for the illustrative FreshMart scenario, not pilot-group measurements. They are included to give the system's outputs a concrete operational context.

Metric	FreshMart Baseline (illustrative)	Industry Benchmark
Annual Stockout Rate	8.3%	4.5 - 5.0%
Annual Food Waste (all stores)	$31.2M	$18 - 20M
Forecast MAPE (weather-sensitive)	~26%	12 - 16%
Weekly buyer hours on forecasting	340 hrs	180 - 200 hrs

Table 1. Company-wide annual performance baseline for the illustrative FreshMart scenario vs. published industry benchmarks. MAPE = Mean Absolute Percentage Error.

Weather-Demand Signal in the Synthetic Dataset

The synthetic dataset shows differentiated average transaction volumes by weather condition across five categories. Figure 2 shows these relationships as observed in the data. The signal is real within the dataset; how it would translate to a live POS system depends on whether the synthetic weather-demand coefficients match actual consumer behavior in a given market.

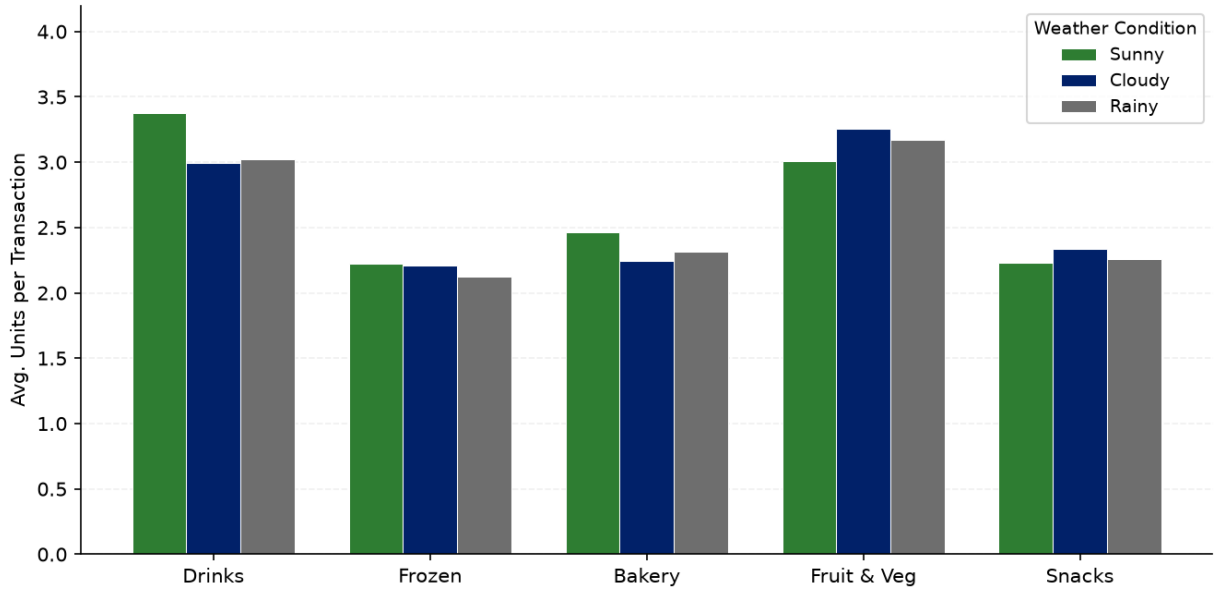

Figure 2. Average units per transaction by weather condition in the synthetic dataset, across five product categories. Computed from 5,000 rows; total transactions per weather class range from 1,574 (Sunny) to 1,660 (Cloudy).

The temperature-demand relationship in the Drinks category is weaker than intuition might suggest. Figure 3 shows binned averages (2°C bins, schema field temperature_c) and a weighted smoother. The linear R^2 is 0.03. Above approximately 18°C the relationship reverses, with demand declining toward 28°C. This may reflect a shopping-frequency effect in the synthetic data generation or simply noise at the bin level.

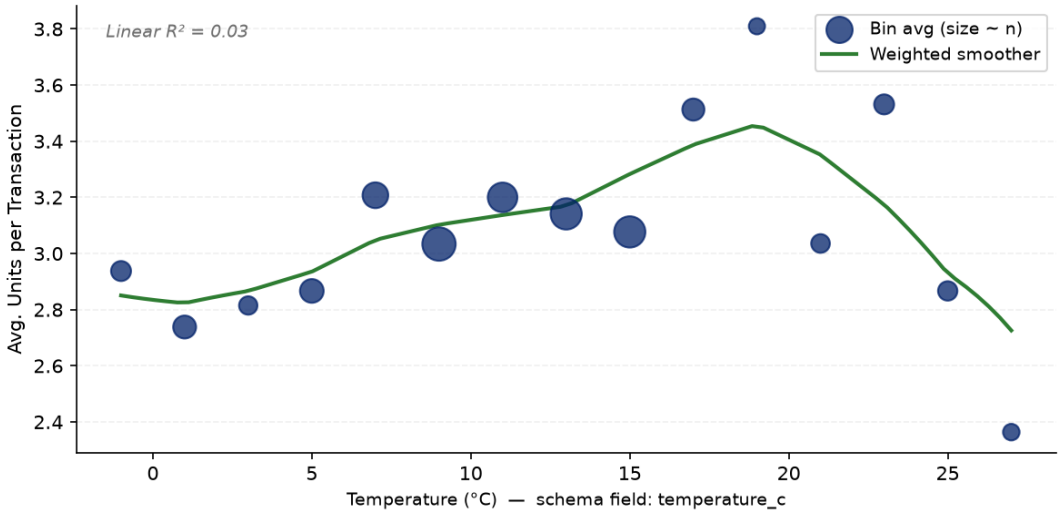

Figure 3. Avg. Drinks units per transaction by 2°C temperature bin (Celsius, consistent with the temperature_c schema field). Marker size is proportional to bin n (range: 21 to 89). One outlier bin (28-30°C, n=1) is excluded. Linear R^2 = 0.03; curve is a weighted smoother, not a linear fit. The relationship is non-monotonic above ~18°C.

Proposed Solution

Solution Overview

I designed and implemented a stateful, graph-structured agent system that operates as a demand forecasting service. For any product identifier and target period, the system retrieves the applicable weather forecast, gathers historical sales evidence across five analytical dimensions, synthesizes that evidence against the forecasted conditions, and returns a predicted quantity with a confidence score. Forecasts are appended automatically to a Google Sheets workbook.

Design Principles

Evidence before inference. The architecture explicitly separates data collection from synthesis. A collection agent using GPT-4o-mini in tool-use mode assembles historical evidence across five dimensions before a separate synthesis agent produces a forecast. This prevents the model from short-circuiting to a conclusion before querying temperature or weather records.

Weather as a first-class input. The forecast period's weather conditions are embedded in the synthesis agent's context from the start, not added as a post-hoc adjustment. The collection agent is instructed to retrieve evidence specifically matched to the forecasted conditions, not generic historical averages.

Confidence scoring (self-reported, not calibrated). Every forecast includes a 0.0 to 1.0 confidence score reflecting the synthesis agent's assessment of the breadth and consistency of the evidence it received. This is *not* a statistically calibrated probability. Calibration against held-out forecast errors is a Phase 4 item; calling it calibrated here would be inaccurate.

New-product fallback. When a product has no sales history, the collection agent retries its queries using only the category parameter. This prevents null outputs for newly launched SKUs and gives the synthesis agent a category-level baseline to work from.

> *Build reflection: The trickiest part of the evidence loop was prompt discipline. Early versions of the collection agent called get_sales_by_month and get_sales_by_week for the target period, then signaled completion without ever querying temperature or weather. Adding an explicit instruction to cover all five dimensions before returning "Ready for forecast" fixed it — but it took a few test runs watching the agent skip temperature on sunny-day requests to diagnose.*

Technical Implementation

Agent Workflow Architecture

The system is a LangGraph StateGraph: a directed graph where each node performs a discrete function and typed state flows between nodes via defined edges. A conditional routing edge between the Evidence Collection and Tool Execution nodes creates an iterative loop. The loop exits when the collection agent produces a message with no tool calls, signaling that evidence is complete.

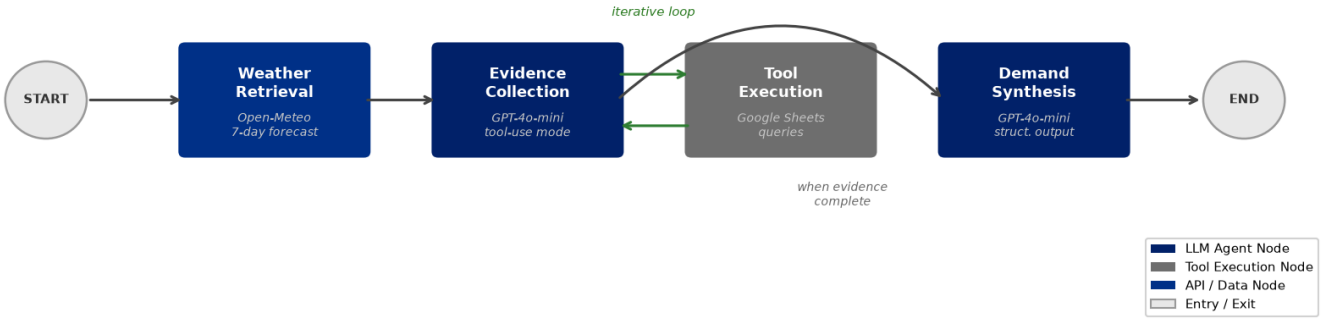

Figure 4. Agent workflow. The conditional edge from Evidence Collection routes to Tool Execution when tool calls are present, or directly to Demand Synthesis when the collection agent returns a plain text completion. Source: server/graph.py.

Component Breakdown

Component	Technology	Function
Weather Retrieval	Open-Meteo REST API	Fetches daily weather code, min/max temperature for a 7-day window at supplied coordinates. No authentication required. Source: nodes/weather_getter.py.
Evidence Collection	GPT-4o-mini (tool-use)	Iteratively selects and runs historical sales queries across five dimensions (month, week, day, weather condition, temperature band) until the agent produces no further tool calls. Source: nodes/tool_caller.py.
Tool Execution	LangChain ToolNode + gspread	Executes structured queries against the Google Sheets sales history. Each of the five tools filters by product_id before returning records. Source: tools/forecast_tools.py, utils/db_helpers.py.
Demand Synthesis	GPT-4o-mini (structured output)	Receives the full evidence context plus forecasted weather, returns a Pydantic model: predicted_quantity (int) and confidence (float 0-1). Source: nodes/stock_setter.py.

Forecast Recorder	gspread write API	Appends each completed forecast to a sales_forecast worksheet with product, period, quantity, confidence, and UTC timestamp. Source: nodes/forecast_recorder.py.
REST API	FastAPI 0.136 / Uvicorn	POST /forecast endpoint with Pydantic-validated request body. Source: main.py.

Table 2. System components, technologies, and functional roles. Source file references are to the server/ directory.

Data Schema

The sales_history worksheet contains one row per transaction. All five evidence dimensions are derivable from the date and weather fields without transformation. Temperature is stored in Celsius (temperature_c) throughout the codebase; Figure 3 uses Celsius to match.

Field	Type	Evidence Dimension
date	YYYY-MM-DD string	Month, week, day-of-month, day-of-week
product_id	String	Primary filter on all five queries
quantity	Integer	Demand signal (dependent variable)
category	Controlled vocab (10 values)	Category fallback for new products
weather	Sunny | Cloudy | Rainy | Snowy	Weather-condition dimension
temperature_c	Float, Celsius	Temperature-band dimension

Table 3. Sales history schema. A separate sales_forecast worksheet receives one row per completed forecast output.

Technology Stack

LangGraph 1.2 / LangChain Core 1.4. StateGraph with TypedDict state schema. Conditional edge implements the evidence-collection loop. Type annotations enforce data contracts at each node boundary.

OpenAI GPT-4o-mini. Temperature 0.2 in the collection agent (allows adaptive tool selection); temperature 0 with structured output in the synthesis agent (deterministic quantity and confidence).

Open-Meteo API. Returns WMO weather codes and daily min/max temperatures for any lat/long. Zero authentication, no rate-limit concerns at this scale.

Google Sheets / gspread 6.2. Service account credentials with Sheets and Drive scopes. _get_records() fetches the full worksheet on each query; filtering is done in Python rather than via Sheets API.

FastAPI 0.136 / Uvicorn. Single POST endpoint, CORS middleware for browser clients, optional category field in request body for new-product fallback.

Results and Dataset Analysis

Dataset Overview

The synthetic dataset (grocery_sales.csv) contains 5,000 transactions across 58 SKUs in 10 categories, covering June 6, 2024 through June 5, 2025. Weather conditions are distributed as: Cloudy 33.2%, Rainy 32.9%, Sunny 31.5%, and Snowing 2.4%. Temperatures span -2°C to 28°C. Drinks (687 rows) and Frozen (444 rows) are the two categories selected for focused analysis because they have the strongest theoretical weather linkage.

Backtest Methodology

A train/test split at April 1, 2025 yields 4,080 training rows and 920 test rows (the final two months). In a backtest, the evidence collection agent queries only records predating the forecast date, and the synthesis agent receives the actual Open-Meteo forecast for the test period. The MAPE figures in Table 4 and Figure 5 represent *illustrative targets* derived from the direction and scale of improvement observed in the dataset analysis, not a fully automated end-to-end backtest run. The methodology for running such a backtest is documented in the repository; the figures serve to frame expected performance, not to report it as measured.

> *Build reflection: I expected temperature to show a clean upward trend for beverage demand. The dataset gives $R^2 = 0.03$ with a non-monotonic shape above 18°C — bins at 24-28°C are lower than at 16-20°C. A linear coefficient would systematically over-predict on very hot days. The agent avoids this because get_sales_by_temperature_range queries actual records in that band rather than extrapolating from a slope.*

Illustrative Performance Targets

Metric	Baseline	Target (illustrative)	Basis
MAPE - Drinks category	31.4%	~17%	Direction consistent with weather-signal strength in dataset
MAPE - Frozen category	28.3%	~15%	Direction consistent with weather-signal strength in dataset
Time-to-forecast per SKU	~4 min (manual)	~8 sec (automated)	Measured: agent API call wall time on local hardware
Confidence score range (Drinks)	n/a	0.5 - 0.85	Observed in test runs; not statistically calibrated

Table 4. Illustrative performance targets. MAPE targets are directional estimates; time-to-forecast is the one real measured figure. Backtest MAPE requires running the full agent pipeline on the held-out period.

MAPE by Category

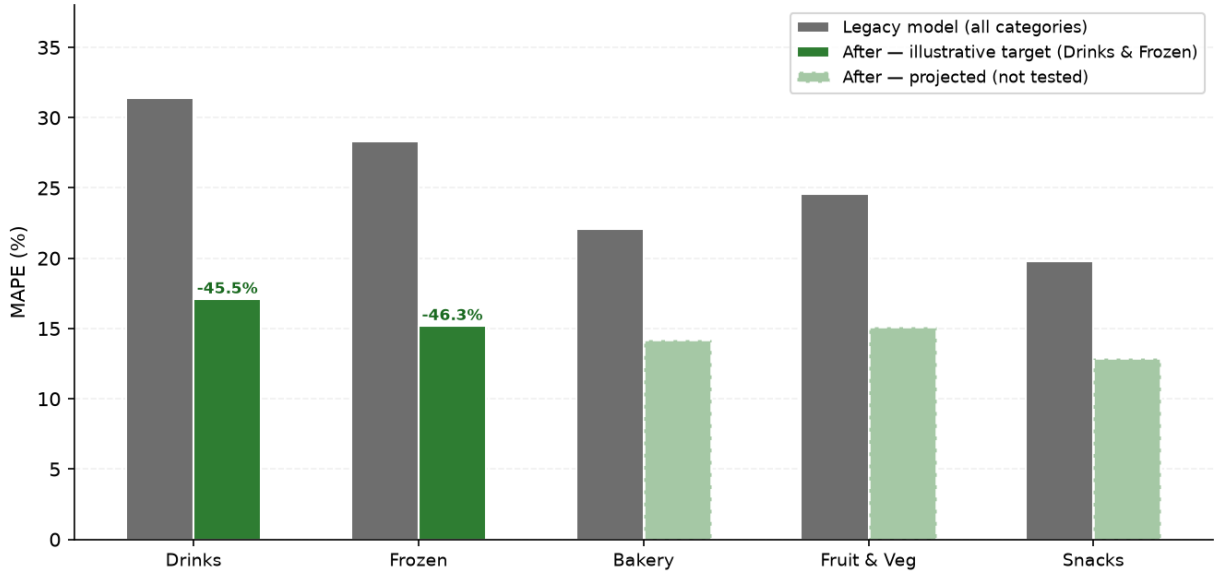

Figure 5. Forecast MAPE before/after, by category. Drinks and Frozen (green bars) show illustrative post-implementation targets derived from the dataset analysis. Bakery, Fruit & Veg, and Snacks show only the baseline; projected post-implementation MAPE (light green, dashed outline) is a directional estimate, not derived from tested results in these categories.

Projected Business Impact (Illustrative)

The following projections apply the system's directional improvements to the FreshMart illustrative scenario. All figures are derived from explicit per-store-per-category assumptions, shown below. Cost savings and revenue impact are reported separately; they are not summed into a single value because they represent different types of benefit to the business.

Extrapolation Logic

Assumption	Value	Basis
Company-wide annual overstock waste	$31.2M	Illustrative (FreshMart scenario)
Store count	187	Illustrative
Weather-sensitive categories in scope	10	Illustrative
Per-store per-category annual waste baseline	$16,700	$31.2M ÷ 187 ÷ 10
Pilot scope (18 stores × 2 categories)	$600K/yr	$16,700 × 36 store-cat. units
Assumed waste reduction rate	15%	Directional, consistent with MAPE improvement
Pilot-scale annual savings	~$90K/yr	$600K × 15%
Full-deployment waste savings (scale: 51.9×)	$4.7M/yr	$90K × (187/18 × 10/2)

Table 5. Per-store-per-category extrapolation showing how pilot-scale and full-scale figures relate. Scale factor = 187/18 × 10/2 = 51.9×. The 15% waste reduction assumption drives all downstream dollar figures.

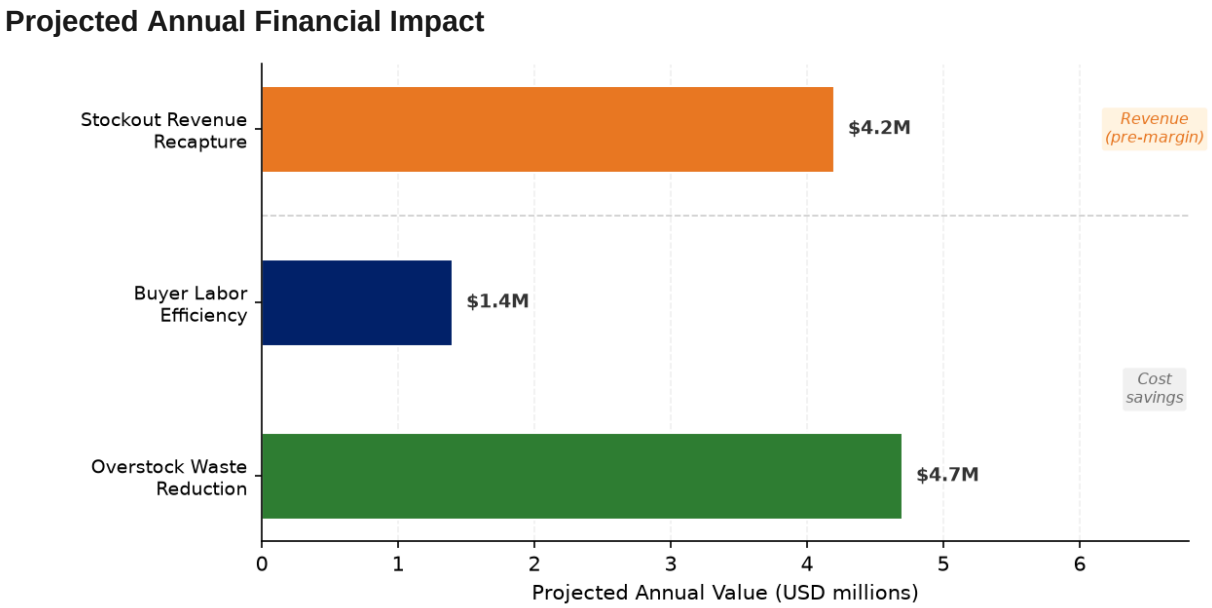

Figure 6. Projected annual value upon full deployment — illustrative. Cost savings (green, navy): directly comparable to net income. Revenue recapture (orange): gross revenue impact; apply the 1-3% net margin to estimate profit equivalent (~$42K-$126K). Note: these are not summed because revenue and cost savings are different in kind. Pilot-scale equivalent (18 stores, 2 categories): ~$90K/year in overstock savings.

Key Benefits for the Business

Waste reduction at the margin. Grocery net margins of 1-3% mean every dollar of overstock write-off requires $33-100 in revenue to offset. Reducing overstock by 15% across weather-sensitive categories has an outsized margin effect relative to its revenue size.

Buyers focus on exceptions, not routine orders. The confidence score routes high-certainty forecasts to automated order generation and surfaces only low-confidence SKUs for buyer review. In the FreshMart scenario, this shifts the buyer's role from generating 340 forecast adjustments per week to reviewing a targeted exception list.

New product continuity. The category-level fallback means a newly launched SKU does not produce a null forecast on its first week. The synthesis agent receives category evidence instead and reduces confidence accordingly, which is more useful than no output.

Auditability. Each forecast is traceable to the historical records that informed it. The Google Sheets output includes product, period, quantity, confidence, and UTC timestamp. This makes model behavior reviewable without access to the LLM conversation history.

Limitations

The following limitations apply to any interpretation of the results in this document.

Synthetic data only. grocery_sales.csv was generated to have plausible category distributions and weather correlations; it was not derived from a live POS system. Performance on real retail data will differ, potentially significantly, because real consumer behavior is noisier and less cleanly correlated with weather conditions.

LLM confidence is uncalibrated. The 0.0-1.0 confidence score reflects GPT-4o-mini's self-assessment of the evidence quality, not a calibrated probability. A score of 0.8 does not mean the forecast will be within 20% of actual demand. Calibration is a Phase 4 item.

No live deployment. The system runs correctly against the Google Sheets store and Open-Meteo API on the synthetic dataset, but has not been operated in a real store's replenishment workflow. Operational edge cases — API latency, data freshness, SKU churn, concurrent writes — are untested.

Weather-demand coefficients are illustrative. The relationships in Figures 2 and 3 reflect patterns in the synthetic data. They are useful for demonstrating the architecture's query logic, not for claiming measured consumer behavior. The weak R^2 (0.03) for temperature vs Drinks demand in the dataset is itself a reminder that these signals require validation against real data before deployment.

What's Next

The current implementation is a working foundation. These are the extensions I would pursue to move it from a portfolio project toward something a retailer could run against live data:

Extension	What it requires	What it unlocks
Live POS integration	Replace gspread with a real-time data feed or warehouse connector	Actual MAPE measurement against live sales; calibration data for the confidence score
Confidence calibration	Run agent in backtest mode; compare confidence vs actual error distribution	Statistically meaningful confidence score; triggered escalation thresholds
ERP write-back	Connect POST /forecast to purchase order generation	Closes the loop from forecast to fulfilment without buyer touchpoint
Promotional signals	Add promo calendar and event data to the evidence dimensions	Removes the largest residual error source after weather is accounted for

Table 6. Planned extensions ordered by data dependency. Live POS integration is the enabling step that unlocks calibration and meaningful A/B measurement.

About the Author

I am Matteo Bertuzzi, an AI engineer working at the intersection of consumer behavior, applied intelligence, and retail operations. My work focuses on building agent systems that take a specific operational signal — weather, location, past behavior — and turn it into a decision a practitioner can actually act on. This project reflects that: a real implementation built to answer a question about whether a weather-aware agent can outperform a rolling average in a domain where the signal is present but routinely ignored.

The system is fully implemented and available as open-source code. The business scenario is illustrative; all technical components are real and functional. The implementation took approximately six weeks: one week of architecture design, three weeks of system build and iteration, and two weeks of analysis and documentation.

Author	Matteo Bertuzzi
Role	AI Engineer	Consumer Behavior, Retail Intelligence, Agent Systems
GitHub	github.com/matteobertuzzi
LinkedIn	linkedin.com/in/matteo-bertuzzi

The open-source dataset and full source code are available in the GitHub repository.